Enterprise Data Reconciliation

From three messy systems to one trusted CFO dashboard

FY2023 USD revenue from currently active customers

Data Analytics Internship · Python · Pandas · SQL

The business problem

A FinTech sells digital products in euros but must report to investors in US dollars

1 Accounts database (SQL)

Every customer and their status. It is a HISTORY of changes, so one customer appears several times.

2 Server logs (text file)

100,000 raw text lines. The purchase details are buried inside free text, and some lines are errors.

3 Exchange rates (CSV)

EUR→USD for each day. Some days have no rate (market closed).

Core question: “How much genuine revenue, in US dollars, did we earn from our currently active customers, how did it change month by month, and which customers are worth the most?”

Genuine

Failed / error events are NOT revenue

In USD

Convert at the rate of the day it happened

Currently active

The LATEST status decides, not an old one

Completeness

No transaction may vanish silently

Solution roadmap: four phases, each with one output

Every phase uses vectorised operations only: no row-by-row loops

1 SQL

Latest status per customer
window function ROW_NUMBER

Output

df_active_users

14,183 customers

2 REGEX

Parse the log text
drop ERROR lines

Output

extracted_data

94,902 transactions

3 MERGE

ffill rates + left join
filter active, convert

Output

df_final

53,899 rows · $148.3M

4 DASHBOARD

Group by month and
by customer, plot

Output

2 charts

Revenue by month · Top 5

Libraries: sqlite3 (database), pandas (tables, regex, merges), matplotlib + seaborn (charts), re / time (utilities)

PHASE 1 · SQL

The accounts table is a history, not a snapshot

What does one customer look like in the database?

user_id	status	updated_at
U-00000	Pending	2023-01-01 10:00
U-00000	Active	2023-01-19 14:00
U-00000	Suspended	2023-02-28 09:00

One customer = three rows. Only the LAST row describes them today.

47,464

rows in the users table

25,000

unique customers

17,582

rows with status Active (ever)

Trap: WHERE status = 'Active' on the raw table counts customers who were Active once but are Suspended now.
Fix: first find each customer's latest row, THEN check whether that row is Active.

CONCEPT

Concept: window functions in plain words

GROUP BY collapses rows. A window function keeps every row and adds a calculated column.

user_id	status	updated_at	rn
U-00000	Suspended	2023-02-28	1 ← latest
U-00000	Active	2023-01-19	2
U-00000	Pending	2023-01-01	3
U-00001	Active	2023-01-28	1 ← latest
U-00001	Pending	2023-01-01	2

```
rn = ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY updated_at DESC)
```

Order of operations: rank first, keep rn = 1, and only then filter status = 'Active'.

PARTITION BY user_id → makes one separate group per customer

ORDER BY updated_at DESC → newest row first inside each group

ROW_NUMBER() → numbers rows 1, 2, 3… so 1 = latest

WITH ranked AS (…) → a CTE: a named temporary result we can filter

Alternative → correlated subquery using MAX(updated_at); same answer, more awkward

PHASE 1 · SQL

Phase 1 solution: SQL for the latest status

One query returns exactly one row per currently Active customer

```
query = '''
WITH ranked AS (
  SELECT user_id, name, status, updated_at,
    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY updated_at DESC) AS rn
  FROM users
)
SELECT user_id, name, status, updated_at
FROM ranked
WHERE rn = 1            -- latest row
  AND status = 'Active' -- is it Active?
'''
conn = sqlite3.connect("enterprise_database.db")
df_active_users = pd.read_sql(query, conn)
```

What happens

1. The CTE numbers each customer's rows, newest = 1.
2. The outer query keeps only rn = 1 (the latest status).
3. It then keeps only those whose latest status is 'Active'.
4. pd.read_sql loads the result into a DataFrame.

Result: 14,183 currently Active customers (of 25,000). Checks: user_id is unique · every status is 'Active' · matches an independent pandas groupby().tail(1) check.

PHASE 2 · REGEX

The server log: data trapped inside text

What does one log line look like, and what separates real revenue from noise?

```
[2023-03-15 00:00:00] INFO: Transaction processed. payload={"u":"U-19484", "item_code":"PRD-874",
"eur_val":3404.44}
[2023-09-09 00:00:00] ERROR: Transaction processed. payload={"u":"U-21438", "item_code":"PRD-793",
"failed_val":41.24}
```

Date | **User ID** | **Product ID** | **Euro value** | ...all inside ONE string

100,000

raw log lines

94,902

INFO lines = real transactions

5,098

ERROR lines = failed events (5.1%)

Why not pd.read_csv? The file has no columns to split, only one text column (Raw_Server_Log).
Rule: drop ERROR lines first, then extract the four fields with a regular expression.

CONCEPT

Concept: regular expressions (regex) in plain words

A regex is a pattern that describes text; capture groups pull out the pieces you want

Symbol	Meaning	Example
\d	one digit 0-9	\d → 7
\d{4}	exactly 4 digits	\d{4} → 2023
[\d.]+	one or more digits or dots	3404.44
[^\]]*	any characters except]	skips ' 00:00:00'
.*?	skip anything, as little as possible	jump to the next key
(?P<name>…)	capture group with a name	becomes a column
\[\. \"	backslash = literal symbol	match [. " exactly

Why named groups? Each group name (Date, User_ID, Product_ID, Euro_Value) becomes a column automatically.

Why vectorised? str.extract applies the pattern to all 94,902 rows in one call, no loop.

Think of the pattern as a template with holes: fixed text is matched, the groups are the holes we keep.

PHASE 2 · REGEX

Phase 2 solution: drop errors, extract, convert types

Output: extracted_data, with 4 clean columns and 94,902 rows

```
logs = pd.read_csv("server_logs.txt")
is_error = logs["Raw_Server_Log"].str.contains("ERROR", regex=False)
valid_logs = logs.loc[~is_error].copy()

pattern = (r"\[(?P<Date>\d{4}-\d{2}-\d{2})[^\]]*\]"
           r".*?\"u\":\"(?P<User_ID>U-\d+)\""
           r".*?\"item_code\":\"(?P<Product_ID>PRD-\d+)\""
           r".*?\"eur_val\":(?P<Euro_Value>[\d.]+)")
extracted_data = valid_logs["Raw_Server_Log"].str.extract(pattern)

extracted_data["Date"] = pd.to_datetime(
    extracted_data["Date"], format="%Y-%m-%d")
extracted_data["Euro_Value"] =
extracted_data["Euro_Value"].astype(float)

assert extracted_data.isna().any(axis=1).sum() == 0
```

Steps

1. Load the log as one text column
2. Boolean mask: drop ERROR lines
3. str.extract with named groups
4. Convert text → datetime / float
5. Assert nothing failed to parse

Date	User_ID	Product	Euro
2023-03-15	U-19484	PRD-874	3404.44
2023-05-09	U-18932	PRD-905	3406.21

Checks: 0 unparsed lines · rows = 100,000 − 5,098 = 94,902 · Date is datetime64, Euro_Value is float64.

PHASE 3 · MERGE

Exchange rates: 104 blank days (28.5%)

What goes wrong if we join transactions to the rates as they are?

Date	Weekday	EUR_to_USD (raw)
2023-01-04	Wednesday	1.1762
2023-01-05	Thursday	1.0883
2023-01-06	Friday	(blank)
2023-01-07	Saturday	(blank)
2023-01-08	Sunday	1.1384

104 of 365

days have no rate: 52 Fridays + 52 Saturdays

67,918

of 94,902 rows survive an inner join

26,984

transactions (28.4%) would vanish silently

Note: the brief talks about weekends, but in this file the blanks are Friday and Saturday. I checked with day names, so the filled rate is Thursday's.

An error-free script can still be wrong: this is the 'completeness' risk in the CFO's question.

CONCEPT

Concept: forward-fill and join types

Two ideas decide whether the revenue table is complete

Forward-fill (ffill)

Date	Before	After ffill
Thu 5 Jan	1.0883	1.0883
Fri 6 Jan	blank	1.0883
Sat 7 Jan	blank	1.0883

Copy the last known rate down. Dates must be sorted first.

Inner join vs left join

Inner join: keeps only rows that match on both sides. Unmatched transactions DISAPPEAR.

Left join: keeps every transaction (left table). No match would show as NaN, which we can detect.

validate="many_to_one": pandas raises an error if the right table has duplicate keys, preventing hidden row multiplication.

Business reasoning: when the market is closed, finance uses the last published rate. That is exactly what forward-fill does.

PHASE 3 · MERGE

Phase 3 solution: fill, merge, convert

Three steps produce df_final with a USD_Revenue column

```
rates = pd.read_csv("daily_exchange_rates.csv")
rates["Date"] = pd.to_datetime(rates["Date"])
rates = rates.sort_values("Date").reset_index(drop=True)
rates["EUR_to_USD"] = rates["EUR_to_USD"].ffill()   # 0 gaps left

# 1) keep ALL transactions: left join on the date
df = extracted_data.merge(rates, on="Date",
        how="left", validate="many_to_one")
assert len(df) == len(extracted_data) and df["EUR_to_USD"].notna().all()

# 2) keep ACTIVE customers only (inner join on purpose)
df_final = df.merge(df_active_users[["user_id", "name"]],
        left_on="User_ID", right_on="user_id",
        how="inner", validate="many_to_one").drop(columns="user_id")

# 3) convert euros to dollars at that day's rate
df_final["USD_Revenue"] = df_final["Euro_Value"] * df_final["EUR_to_USD"]
```

Why each choice

to_datetime: text dates do not join reliably

sort + ffill: 'last known' needs time order

left join: no valid transaction can drop out

inner join on active users: deliberate filter, counted later

validate: blocks duplicate-key fan-out

vectorised multiply: one line for all rows

Result: df_final = 53,899 rows · 41,003 non-active transactions excluded on purpose.

PHASE 3 · MERGE

Worked example: following two transactions end to end

Seeing one row travel through the pipeline makes the logic easy to explain

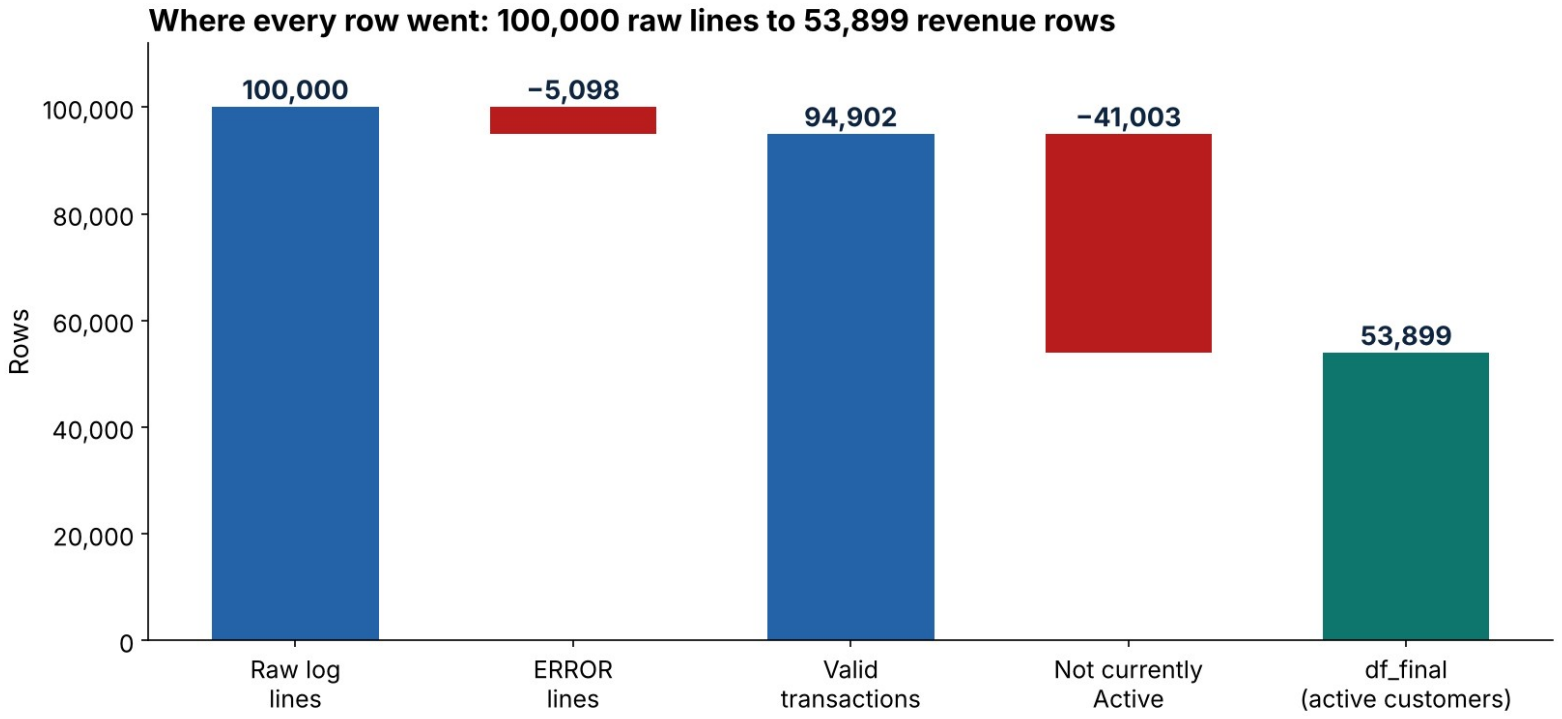
A · Normal day (Wednesday)		B · Saturday (no published rate)	
Log line	[2023-03-15] … U-19484, PRD-874, eur_val 3404.44	**Log line**	[2023-10-21] … U-15882, PRD-284, eur_val 164.08
Active?	Yes (latest status Active)	**Active?**	Yes (latest status Active)
Rate on 15 Mar	1.1782 (published)	**Rate on 21 Oct**	blank → ffill → Thursday 19 Oct = 1.0753
USD revenue	3,404.44 × 1.1782 = $4,011.22	**USD revenue**	164.08 × 1.0753 = $176.44

Without forward-fill, transaction B would have been silently dropped.

PHASE 3 · MERGE

Reconciliation: every row is accounted for

An audit report must explain every difference between input and output



All valid EUR: €237.8M

Active customers EUR: €135.0M

Active customers USD: $148.3M

Average rate: 1.0987

Customers missing from DB: 0

The code asserts 94,902 − 41,003 − 0 = 53,899. Monthly totals also add up exactly to the grand total.

PHASE 4 · DASHBOARD

Phase 4: aggregate and plot

groupby does the maths; matplotlib draws two side-by-side charts

```
monthly = (df_final.groupby("Month", as_index=False)
           ["USD_Revenue"].sum())
top5 = (df_final.groupby(["User_ID", "name"], as_index=False)
        ["USD_Revenue"].sum().nlargest(5, "USD_Revenue"))

# monthly totals must equal the grand total
assert abs(monthly.USD_Revenue.sum()
           - df_final.USD_Revenue.sum()) < 1e-6

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
axes[0].bar(monthly["Label"], monthly["USD_Revenue"])
axes[1].barh(top5["User_ID"][::-1], top5["USD_Revenue"][::-1])
# + titles, axis labels, $ formatters, value labels
```

Design choices for the CFO

Clear titles and labelled axes

Values written on every bar

Money shown as $M / $K, not raw numbers

Bars start at zero (honest scale)

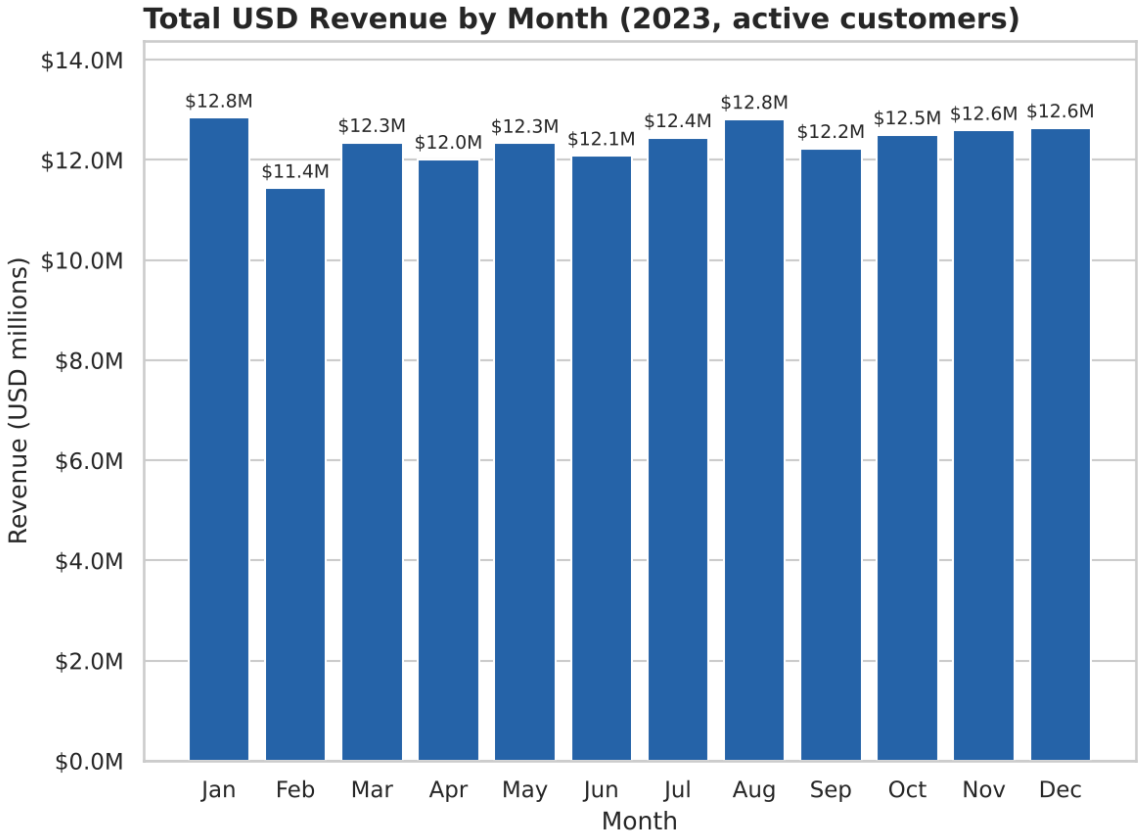
Horizontal bars so customer IDs stay readable

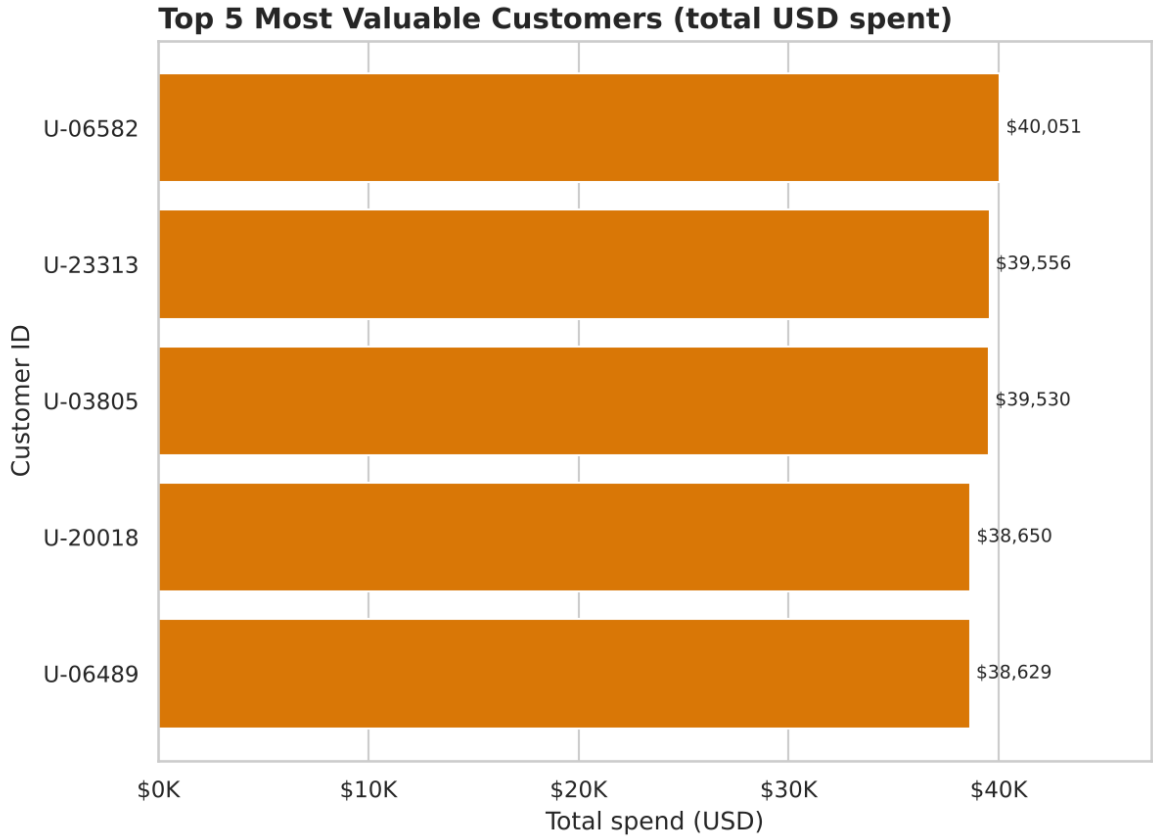
groupby + sum + nlargest = three vectorised steps. No loops anywhere.

PHASE 4 · DASHBOARD

The CFO dashboard

Total USD revenue by month and the top 5 most valuable customers (active customers only)





Top 5: U-06582 ($40,051) · U-23313 ($39,556) · U-03805 ($39,530) · U-20018 ($38,650) · U-06489 ($38,629)

What the data tells the CFO

Four findings in plain language

$148.3M

Total revenue

53,899 transactions from 13,888 active customers (about 3.9 each). Average transaction: $2,751.

≈25% per quarter

Very stable

Q1 $36.6M · Q2 $36.4M · Q3 $37.5M · Q4 $37.7M. No strong trend or seasonality.

+9.9%

FX effect

Converting at daily rates gives $13.3M more than the euro amount ($135.0M → $148.3M).

0.13%

Concentration

The top 5 customers together are 0.13% of revenue: no dependency on any single customer.

Why the weekend fix mattered: 28.4% of transactions (≈$42.1M of revenue) happened on Friday/Saturday, which have no published rate.

Efficiency: why the notebook runs in about 1 second

Requirement: under 2 minutes on 100,000 rows with no row-by-row loops

Task	Loop approach (not allowed)	What I used
Latest status per customer	Loop over users, find max date	SQL window function
Parse 100,000 lines	iterrows + re.search per line	str.extract (one call)
Drop errors	if 'ERROR' in line	Boolean mask
Fill missing rates	Loop and copy previous	ffill()
Join rates	Look up each date in a loop	merge()
USD conversion	Loop and multiply	Column × column

1.2 s

total runtime for the full notebook (limit: 120 s)

Vectorised = the operation runs on the whole column at once, in fast compiled code.

Scales to 10× the data: the SQL window function runs inside the database and everything else is vectorised.

How I checked my own work

Professional analysts verify before they trust

Check	Result
SQL latest status vs independent pandas groupby().tail(1)	Identical set of 14,183 customers
user_id unique in df_active_users	True (assert passes)
Rows: raw 100,000 − 5,098 errors = 94,902	Matches extracted_data (assert)
Lines that failed regex parsing	0
Missing rates after ffill	0
Row count after left join with rates	94,902 (unchanged)
94,902 − 41,003 − 0 unknown customers	= 53,899 rows in df_final
Monthly revenue sums to the grand total	True within 1e-6
Euro_Value > 0, no duplicate log lines	True / 0 duplicates

Assumptions, limitations and next steps

Being honest about what the analysis does and does not claim

Assumptions and limits

Currently active = latest status today, not status at purchase time.

Forward-fill (last published rate) stands in for closed-market days.

All log amounts are in EUR, as the brief states.

Data covers 2023 only; the brief's 'Q1-Q4' means the four quarters of 2023.

41,003 transactions from non-active customers are excluded by design.

Possible next steps

Report revenue from non-active customers separately so the CFO sees the whole picture.

Use status at the time of each purchase (as-of join) if the business wants that definition.

Turn the notebook into a scheduled, tested pipeline.

Add data-quality alerts (missing rates, parse failures).

Add a quarterly view and product-level revenue.

Key terms in one place

Quick definitions to use during the presentation

Term	Meaning
CTE (WITH …)	A named temporary result inside a SQL query
Window function	Calculates a value per row using related rows, without collapsing them
ROW_NUMBER / PARTITION BY	Numbers rows 1, 2, 3 inside each group (here: each customer)
Regex / capture group	A text pattern / the part of the match you keep as a column
Vectorised operation	Works on a whole column at once, not row by row
Forward-fill (ffill)	Copy the last known value into the blanks that follow
Inner vs left join	Only matches kept vs every left-table row kept
Reconciliation	Showing that the numbers from each step add up
Assert	A check that stops the code with an error if it is false

Questions teachers may ask (1/2)

Short, honest answers

Q: Why not just filter status = 'Active'?

Because the table is a history. A customer who was Active in January but Suspended in March would be counted wrongly. We rank by updated_at and use only the latest row.

Q: Why a window function and not GROUP BY?

GROUP BY collapses rows and loses the status column. A window function keeps the rows and lets us pick the latest one.

Q: Why drop ERROR lines before the regex?

They have failed_val instead of eur_val, they are not revenue, and the pattern would not match them.

Q: Why forward-fill and not delete or average?

Finance practice is to use the last published rate. Deleting would lose 28% of transactions; an average would invent a rate.

Questions teachers may ask (2/2)

Short, honest answers

Q: Why a left join with rates but an inner join with active users?

The left join protects completeness (every valid transaction must keep a rate). The inner join is a deliberate filter to active customers; the dropped rows are counted.

Q: What does validate='many_to_one' do?

It raises an error if the right-hand table has duplicate keys, which would silently multiply rows.

Q: How do you know the SQL result is right?

I rebuilt it independently in pandas with sort_values + groupby().tail(1); the customer sets were identical.

Q: Would this work with 10× more data?

Yes. The window function runs in the database and the rest is vectorised, so it still takes seconds.

Summary

SQL window function → 14,183 currently Active customers.

Regex on 100,000 log lines → 94,902 genuine transactions (5,098 errors removed).

Forward-fill + left join → no transaction lost to missing rates; converted to USD at each day's rate.

Result: $148.3M revenue from 53,899 transactions, stable at $11.4M–$12.8M a month, and every row reconciled.

Thank you · Questions?